The CKS-vs-External-Structured-Memory Boundary as Standalone Architectural Commitment: Adjacency 3 in the Coordination Knowledge Substrate Pattern

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: 5 May 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize the third of the three adjacencies the parent foundational note A1.14 commits to — the boundary between CKS and external structured memory of the Knowledge Objects (KO) and OIDA family — as a standalone architectural specification, separable from the integrating-frame three-adjacencies treatment in A2.81 and from the inheritance-side treatment in A1.09 with which it stands in complementary relation.

Abstract

The CKS pattern's three-adjacencies commitment per A1.14 names external structured memory of the KO/OIDA family as the third architectural neighbor against which CKS must be distinguished. The integrating-frame note A2.81 names this adjacency at the integrating level; the inheritance-side treatment in A1.09 names what CKS shares with the family. Neither treatment is sufficient on its own for the standalone defensive specification this note supplies. The boundary operates on a specific architectural distinction: design posture rather than feature delta. CKS sits in the same architectural family as KO and OIDA — typed external substrate, separation of concerns between storage and processing, addressable units with provenance — and inherits the database-like cost model from this family. What distinguishes CKS within the family is four architectural commitments that together characterize a different design posture: human governance as design requirement (§3.1, §5.2 of the source paper), formal role and authority schema at the cell level (§5.2), AI's architectural role as substrate mediator rather than substrate client (§5.2, §8.2), and where disambiguation work lives — at human-authored schema time rather than at runtime via algorithm (§6.2). This note states the four axes as the operational components of the boundary, distinguishes the boundary from adjacent patterns commonly conflated with it, names the failure modes that violate the boundary by collapsing the architectural difference into a feature delta, and provides an operational test for whether a system's architectural posture distinguishes it from external structured memory of the KO/OIDA family.

1. Why the CKS-vs-KO/OIDA boundary needs to be formalized as standalone

The parent foundational note A1.14 commits to three adjacencies CKS holds against in the surrounding 2026 literature: CKS is not RAG, not parametric memory, and not external structured memory in the style of Knowledge Objects (KO) and OIDA. The integrating-frame note A2.81 names all three at the integrating level. A2.82 formalized Adjacency 1 (RAG) as standalone; A2.83 formalized Adjacency 2 (parametric memory) as standalone. This note formalizes Adjacency 3 — the CKS-vs-KO/OIDA boundary — as standalone, with particular weight on the four distinguishing axes that specify a different design posture rather than a feature delta.

Three motivations sit behind the standalone framing. First, the conflation patterns are concrete and recurrent. Evaluations sometimes treat CKS as "KO with extra fields," missing that the extra fields do disambiguation work CKS's design philosophy requires to be explicit rather than computed (§6.2 of the source paper). Analyses sometimes treat CKS as "OIDA plus multi-human use," missing the architectural-difference-vs-feature-addition distinction the source paper deploys at §5.2 and reaffirms at §9.4. Deployments sometimes present structured memory with policy attachment as CKS-equivalent, missing the AI-as-substrate-mediator commitment per A1.04 and the formal role/authority schema commitment per A2.46 and A2.47. Second, the boundary is the most consequential of the three adjacencies for prior-art positioning, because CKS shares the architectural family with KO and OIDA per A1.09's inheritance treatment; many architectural features are common to CKS and the family it inherits from, and the four distinguishing axes are what defensibly differentiate CKS within the family. Third, the boundary stands in complementary relation to A1.09: A1.09 specifies what CKS inherits from KO and OIDA along the multi-human axis; A2.84 specifies what distinguishes CKS at the architectural-posture level. Same family per A1.09, different design posture per A2.84. A2.53 within A1.09's decomposition operationalizes the boundary between adjacency and inheritance for the OIDA case as the architectural-difference-vs-feature-addition claim; A2.84 specifies the boundary in adjacency terms.

2. External structured memory (KO/OIDA family), defined precisely

External structured memory is a family of architectures — represented in the 2026 literature by Zahn and Chana's Knowledge Objects (KO) and by OIDA — in which knowledge is stored as typed structured units in an external store the LLM reads as a substrate client. KO encodes facts as hash-addressed Knowledge Objects with a `provenance_metadata` field, achieves O(1) retrieval at near-constant per-query token cost, and demonstrates the database-like cost curve empirically — at N=7,000, ~252× lower per-query cost than in-context memory, with cost differences that grow linearly as the corpus expands (§6.2). The architectural commitment is to retrieval optimization through structured addressing.

OIDA structures organizational knowledge as typed Knowledge Objects carrying epistemic class (fact, hypothesis, decision, question), importance scores with class-specific decay, and signed contradiction edges between conflicting objects, with a deterministic Knowledge Gravity Engine maintaining the substrate over time (§5.2, §8.2). The architectural commitment is to substrate maintenance through deterministic computation.

Across KO, OIDA, and similar architectures, the shared family commitment is that coordination-grade knowledge is a schema-level object with typed structure and maintained state, not a retrieval target recovered from a corpus at inference time. This shared commitment distinguishes the family from RAG (Adjacency 1, treated in A2.82) and from parametric memory (Adjacency 2, treated in A2.83). Within the family, the LLM operates as a substrate client that reads structured memory to answer queries; substrate maintenance and retrieval optimization are addressed through deterministic computation, with the LLM positioned as consumer rather than as mediator over the substrate's human-facing state.

3. The four distinguishing axes — the CKS-vs-KO/OIDA boundary, defined precisely

The CKS-vs-KO/OIDA boundary specifies four operational components — four axes on which CKS commits to a different design posture than the rest of the family takes. Each is independently load-bearing for the boundary; failing any one collapses the architectural difference into a feature delta.

Axis 1 — Human governance as design requirement. CKS commits to human authority over substrate content as a property of the architecture per A1.01 (§3.1 of the source paper). Humans retain the three rights — to inspect, to modify, and to override substrate content and orchestration rules — at all times during the substrate's existence per A2.01–A2.03, within authority scope per A2.47. KO and OIDA commit to maintenance architectures that operate over the substrate deterministically without locating authority in any particular actor: KO's hash addressing maintains addressability and retrieval correctness; OIDA's Knowledge Gravity Engine maintains substrate state over time through deterministic computation. Neither commits to humans as the actor who holds authority over substrate content (§5.2). The architectural commitment differs on whether authority is a first-class architectural object (CKS) or a maintenance-layer concern (KO/OIDA).

Axis 2 — Formal role and authority schema at the cell level. CKS encodes who may decide what, on what evidence, under which policy, as substrate content the cell's orchestration rules draw on per A2.46 (Category 4 source-of-truth, the substrate as authoritative for "what rules apply") and A2.47 (Category 5 source-of-truth, the substrate as authoritative for "who has what authority"). The role/authority schema is substrate content with the same architectural commitments as other substrate content — addressable, persistent, governance-bound. KO commits to a single `provenance_metadata` field on each Knowledge Object — provenance metadata is captured at the retrieval-unit level but not architecturally extended into a role/authority schema. OIDA's Knowledge Objects carry epistemic class (fact, hypothesis, decision, question), but epistemic class is a characterization of knowledge, not institutional provenance in the role-authority sense; OIDA does not commit to a role/authority schema (§5.2). The architectural commitment differs on what the substrate's schema carries: epistemic characterization or attribution metadata in KO/OIDA; formal role and authority semantics as substrate content in CKS.

Axis 3 — AI's architectural role. CKS frames AI as mediator over the substrate per A1.04 — a five-property commitment formalized in the mediator decomposition A2.18–A2.23. The LLM reads substrate as primary source of state (Property A per A2.19), writes under orchestration rules (Property B per A2.20), does not hold substrate-relevant state outside the substrate (Property C per A2.21), does not exercise authority over substrate content (Property D per A2.22), and outputs that affect substrate

state are recorded with attribution (Property E per A2.23). KO and OIDA frame AI as substrate client consuming structured memory to answer queries — the LLM uses the structured memory as its source of facts, and the mediator role is not committed to architecturally (§5.2, §8.2). The architectural difference is whether AI operates as substrate mediator under human-authored orchestration rules (CKS) or as substrate client recovering structured facts (KO/OIDA).

Axis 4 — Where disambiguation work lives. KO disambiguates computationally at retrieval time — density-adaptive retrieval with learned thresholds (τ=0.85 in KO's experiments) and exact structured key matching on `(subject, predicate)` tuples as fallback. The disambiguation algorithms operate at runtime on substrate queries. CKS disambiguates through human-authored schema at the cell level — encoding distinctions into substrate structure at authoring time per A2.66 rather than recovering them by runtime algorithm (§6.2). The architectural commitment differs on when disambiguation work happens (runtime in KO; authoring time in CKS) and on who does it (algorithm in KO; human author in CKS).

The four axes together define the CKS-vs-KO/OIDA boundary architecturally. They are independent commitments that together characterize a different design posture, not increments on top of the design posture KO and OIDA take. A system that satisfies all four has the boundary in the architectural sense; a system that satisfies fewer has only a feature-level approximation of it.

4. What the boundary does NOT claim, and what it is NOT

The standalone treatment is bounded; stating its limits and distinguishing it from adjacent patterns keeps the framing precise.

The boundary does not claim KO/OIDA is inferior to CKS. KO addresses retrieval-cost optimization at scale through hash addressing; OIDA addresses substrate maintenance through the Knowledge Gravity Engine. CKS addresses coordination-and-decision-layer commitments through human-governed substrate. Each line serves a different design goal coherently on its own terms; the boundary specifies architectural distinction in design posture, not architectural superiority.

The boundary does not deny that CKS inherits from the family. CKS inherits the typed external substrate architecture, the separation of concerns between storage and processing, addressable units with provenance, and the database-like cost model from KO and OIDA per A1.09 and the inheritance decomposition A2.49–A2.54. The four distinguishing axes specify what makes CKS's design posture different *within* that shared family. The inheritance and the adjacency treatments are complementary, not contradictory.

The boundary does not foreclose CKS substrates from including KO/OIDA-style features. A CKS substrate may include hash-addressed entries, importance scores, signed contradiction edges, or other family features; the architectural commitment is to the four distinguishing axes operating, not to exclusion of family features. Nor does the boundary require complete absence of governance from KO/OIDA deployments. Specific deployments may include governance features at the deployment layer; the architectural commitment is to whether the architecture *itself* commits to human governance, role/authority schema, and AI-mediator role as design requirements. Deployment-level governance

attachments do not transform KO or OIDA into CKS architecturally.

The boundary does not foreclose CKS-composed-with-KO/OIDA hybrid compositions. Per A2.85 and the hybrid systems composition framework A1.16, CKS may compose with external structured memory in hybrid systems where the structured memory holds typed facts and contradictions at scale, while the CKS layer adds human governance, role/authority schema, and substrate-mediator semantics. The boundary specifies what each layer is for; the hybrid uses both layers explicitly. The boundary is what makes the explicit naming possible.

The boundary is also not equivalent to four adjacent patterns commonly conflated with CKS within the structured-memory tradition. **Not "KO with extra fields":** the extra fields are not decoration; they are substrate content doing disambiguation work CKS's design philosophy requires to be explicit rather than computed (§6.2). The architectural difference is at the design-posture level (where disambiguation work lives), not at the field-count level. **Not "OIDA plus multi-human":** CKS is architecturally different from OIDA on the multi-human axis, not OIDA plus multi-human (§5.2, §9.4); A2.53 specifically formalizes this architectural-difference-vs-feature-addition claim. **Not hybrid KO/OIDA-with-governance-attached:** operational governance features can coexist with KO or OIDA architectures while preserving their KO/OIDA architectural posture; transformation to CKS requires the four distinguishing axes operating at the architectural level, not at the deployment-feature level. **Not structured memory with policy attachment:** policy attachment improves what the architecture enforces; the four distinguishing axes specify what the architecture commits to at the design-posture level. The two operate at different scopes.

5. Why the boundary is load-bearing for downstream commitments

The CKS-vs-KO/OIDA boundary is load-bearing for several CKS commitments downstream. The integrating three-adjacencies specification per A1.14 and A2.81 depends on this boundary being formalizable as standalone; without it, CKS would be conflated with KO or OIDA at the most architecturally proximate adjacency, and the integrating frame would lose its third axis.

The KO/OIDA inheritance treatment per A1.09 and the inheritance decomposition A2.49–A2.54 stands in complementary relation to this note. Together, the inheritance and adjacency treatments characterize CKS's position in the architectural landscape — same family per A1.09, different design posture per A2.84. A2.53 within A1.09's decomposition operationalizes the boundary between adjacency and inheritance as the architectural-difference-vs-feature-addition claim; A2.84 specifies the boundary in adjacency terms with the four operational components.

The boundary is also load-bearing for several foundational CKS commitments. The human-governed commitment per A1.01 distinguishes CKS from KO/OIDA on Axis 1; the AI-as-substrate-mediator commitment per A1.04 and the mediator decomposition A2.18–A2.23 distinguishes CKS from KO/OIDA on Axis 3; the Category 4 source-of-truth per A2.46 and Category 5 source-of-truth per A2.47 distinguish CKS from KO's `provenance_metadata` field and OIDA's epistemic class on Axis 2; the schema-authoring commitment per A2.66 distinguishes CKS from KO's runtime disambiguation on Axis 4. Each is a foundational commitment of which the boundary specifies the

KO/OIDA-side counterpart.

The hybrid systems composition framework per A1.16 supports CKS-composed-with-KO/OIDA compositions through its three patterns. A2.85 specializes hybrid composition coherence; this note formalizes the boundary that makes the layers nameable as architecturally distinct in any such hybrid. Together, A2.84 and A2.85 close the decomposition of A1.14 by specifying both what the boundary is and what its preservation requires under composition.

6. Failure modes that violate the boundary

A system can fail the CKS-vs-KO/OIDA boundary specifically by collapsing one or more of the four distinguishing axes into a feature delta. Six failure modes name the most common ways this happens.

(a) Substrate-as-extended-KO. The implementation treats the substrate as KO with extra fields — adding fields to hash-addressed Knowledge Objects without committing to the four distinguishing axes. Axis 4 fails because disambiguation remains computational at retrieval time rather than human-authored at schema time. The architectural difference is collapsed into a field-count delta, and the design-posture commitment is lost.

(b) Substrate-as-OIDA-plus-multi-human. The implementation treats CKS as OIDA with multi-human features added — combining OIDA's Knowledge Gravity Engine and epistemic class with multi-user access controls. The architectural-difference-vs-feature-addition distinction per A2.53 is violated; the commitment to a different design posture is reduced to multi-human-feature addition on top of OIDA's existing posture. The phrase the source paper deploys at §5.2 and §9.4 — *architecturally different from OIDA on the multi-human axis, not OIDA plus multi-human* — is what this failure mode collapses.

(c) AI-as-client-in-CKS. The implementation operates the substrate with AI consuming structured content as queries — the LLM is positioned as substrate client rather than substrate mediator. Properties A through E per A2.19–A2.23 fail; Axis 3 collapses, and the AI-as-substrate-mediator commitment per A1.04 is reduced to an unrealized aspiration.

(d) Provenance-metadata-as-role/authority-schema. The implementation treats KO's `provenance_metadata` field as equivalent to CKS's role/authority schema. Axis 2 fails because the schema does not encode who may decide what, on what evidence, under which policy, as substrate content per A2.46 and A2.47; provenance metadata at the retrieval-unit level is not the same architectural object as a formal role/authority schema at the cell level.

(e) Knowledge-gravity-engine-as-governance. The implementation treats OIDA's Knowledge Gravity Engine as equivalent to CKS's human governance. Axis 1 fails because the engine operates over substrate deterministically without locating authority in any particular actor — maintenance-architecture-deterministic-computation is not the same architectural object as human-authority-as-design-requirement. The same failure pattern applies when OIDA's epistemic class is treated as equivalent to authority structure: epistemic class characterizes knowledge, not institutional provenance in the role-authority sense.

(f) Hybrid presented as unified. The implementation runs CKS alongside KO or OIDA but presents the hybrid as a single CKS architecture rather than as an explicit composition per A2.85 and A1.16. The boundary is obscured because the layers are not named, the structured-memory layer's commitments cannot be distinguished from the CKS layer's commitments, and the four distinguishing axes operate on neither layer cleanly. This is not a violation of any single axis but of the conditions under which the boundary remains defensible at all in compositional settings.

A system that exhibits any of (a)–(f) does not instantiate the CKS-vs-KO/OIDA boundary in the architectural sense, even when it operationally combines substrate-style features with KO/OIDA-style features.

7. Operational test, one-sentence test, and closing

A system instantiates the CKS-vs-KO/OIDA boundary if and only if all of the following are true at all times during the substrate's existence.

1. Human governance is a first-class architectural commitment — humans hold authority over substrate content per A1.01, with the three rights per A2.01–A2.03 architecturally exercisable within authority scope per A2.47 (Axis 1).

2. A formal role/authority schema is substrate content — who may decide what, on what evidence, under which policy, is encoded as substrate content per A2.46 and A2.47 (Axis 2).

3. AI operates as substrate mediator — Properties A through E per A2.19–A2.23 hold for LLMs in cells (Axis 3).

4. Disambiguation work lives at human-authored schema level — distinctions are encoded into substrate structure at schema-authoring time per A2.66, not recovered by runtime algorithm (Axis 4).

5. The architecture inherits from the KO/OIDA family per A1.09 where appropriate — typed external substrate, separation of concerns, addressable units with provenance, database-like cost model — while distinguishing on the four axes.

6. Hybrid compositions with KO or OIDA per A2.85 and A1.16 explicitly name the layers: the structured-memory layer holds typed facts and contradictions at scale; the CKS layer adds human governance, role/authority schema, and substrate-mediator semantics.

A system that fails any of (1)–(6) does not instantiate the boundary architecturally, even if it operationally combines substrate-style and KO/OIDA-style features.

A one-sentence test, derived from §1.2, §5.2, and §8.2 of the source paper, supplies a quick classifier for any specific instance: if the substrate's design treats AI as a client consuming structured memory to answer queries, with no first-class commitment to human authority over substrate content or to a formal role/authority schema at the cell level, it is external structured memory of the KO/OIDA family; if the substrate is human-governed by architectural commitment, encodes role and authority semantics at the cell level, and treats AI as substrate mediator under human-authored orchestration rules, it is CKS. The

four-component specification in §3 above provides the full architectural definition for cases requiring detailed analysis.

Implementations under pressure to position AI-coordination architectures within the structured-memory tradition consistently drift toward "KO with extra fields" or "OIDA plus multi-human" presentations, because the surrounding 2026 literature is well-developed and adding incremental features to existing architectures appears as natural extension. Implementations that drift away from the boundary produce systems where CKS is architecturally indistinguishable from advanced KO or OIDA variants, and where the commitments per A1.01 and A1.04 are obscured precisely because the boundary is unclear. Naming the boundary as standalone — with the four axes in §3, the limitations in §4, the load-bearing connections in §5, the six failure modes in §6, and the operational and one-sentence tests in this section — gives downstream readers a precise specification of what distinguishes CKS from external structured memory of the KO/OIDA family. The companion note A2.85 specializes hybrid composition coherence; together with this Adjacency 3 specification, A2.85 closes the decomposition of A1.14.

Subsequent work that adopts the CKS pattern, extends it, composes it with adjacent patterns, or argues against it should use "the CKS-vs-KO/OIDA boundary" in the sense formalized here. Subsequent work that uses the boundary differently is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

Companion notes in this series

Li, W. (2026). *Authority, Not Labor: A Precise Definition of "Human-Governed" in the Coordination Knowledge Substrate Pattern.* 24 April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *Three Adjacencies: Why CKS Is Not RAG, Not Parametric Memory, and Not External Structured Memory.* 24 April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *Inheritance and Extension: How CKS Builds on Knowledge Objects and OIDA Along the Multi-Human Axis.* 27 April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *The CKS-vs-External-Structured-Memory Boundary as Standalone Architectural Commitment: Adjacency 3 in the Coordination Knowledge Substrate Pattern.* 5 May 2026. ORCID: 0009-0004-8065-3235.